Simulator 6: Structured Data Extraction

Alignment: Domains 4 and 5 · Runs on the Claude Agent SDK through your Claude login, against mock backends

Objective

Push seven invoices, receipts and credit notes through an `extract_invoice` tool on the Claude Agent SDK, let the schema stop syntax errors while your own check catches a total that does not add up, retry with the exact validation error, recognise the one document where a retry cannot help, run the same set through a locally simulated batch workflow with a `custom_id` per document, route doubtful records to a human, and read accuracy by field and by document type.

Scenario (official exam scenario 6)

"You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems."

Primary domains: Prompt Engineering & Structured Output, Context Management & Reliability.

Prerequisites

- `uv` and a Claude login (`claude auth login`). Run `uv sync` once in the `labs` folder. No API key.
- Open this simulator in your browser with `uv run serve.py` from `labs`, or run the commands below from this simulator's folder.

What is in the folder

File	Role
`invoice.schema.json`	The `extract_invoice` tool with `strict: true`: every field required, nullable `due_date` and `issue_date`, `currency` and `category` enums with `other` + detail and `unclear`, `stated_total` beside `calculated_total` and `conflict_detected`, `missing_information`, and a per-field `confidence` object
`fixtures/invoices.txt`	DOC-01 to DOC-07: a DD/MM date, a prose invoice, a receipt whose total includes GST, Thai baht, a credit note with no currency, a licence whose line detail lives in an attachment
`fixtures/labels.json`	Ground truth for accuracy by field and type
`extract.py`	The pipeline: sync path with validation retries, batch path with resubmission, routing, accuracy
`cases/UC-*.json`, `cases.py`	Six practice-test use cases and the code that runs them
`verify.py`	Sixteen acceptance checks plus one per use case

Guided tour

1. Read the tool. Count the nullable fields, find the two enum escape hatches, and note why `strict: true` needs `additionalProperties: false` and a complete `required` list.

```
uv run python -m json.tool invoice.schema.json
```

© 2026 Tertiary Infotech Academy · UEN 201200696W

2. Run the sync path. Each document becomes one model turn through the Agent SDK, with the tool and a system prompt that names it and states the normalisation rules. Your Python validator then checks the record. Watch DOC-03 go to a human with both totals kept, and DOC-07 name what's missing and go to a human with no retry spent.

```
uv run extract.py
```

3. Try the exam's `tool_choice` options. On the Messages API, `any` means Claude must call a tool and `forced` names the tool. The Agent SDK has no such setting, so these runs ask for it in the prompt. Read what Claude did, and the lesson in `evidence/extraction.json`. An instruction isn't a guarantee, so always check that a tool call came back.

```
uv run extract.py --tool-choice any --max-docs 2
uv run extract.py --tool-choice forced --max-docs 2
```

4. Open DOC-01. Attempt 1 may return `02/11/2026` for `due_date`; the schema pattern rejects it; attempt 2 receives the document, the failed extraction and the exact error inside `<validation_errors>`, and returns `2026-11-02`. If your run got it right first time, read the retry prompt in `user_content()` instead.

```
uv run python -m json.tool evidence/extraction.json | grep -n -A14 '"custom_id": "DOC-01"' |
    head -40
```

5. Open DOC-03. Line items sum to 35.00; the printed total is 38.00 because GST is not itemised. The schema passed; your semantic check set `conflict_detected`, the retry decision says `stop` because the gap is real data, and the record went to human review.

```
uv run python -m json.tool evidence/extraction.json | grep -n -B3 -A3 'line items sum to 35'
```

6. Open DOC-07. The line detail lives in "schedule A", which was never supplied. The model fills `missing_information` and the record goes to a human, with no retry spent, because no prompt change can recover information that is absent.

```
uv run python -m json.tool evidence/extraction.json | grep -n -A3 '"custom_id": "DOC-07"'
```

7. Run the batch workflow. It's simulated locally through the Agent SDK: one `custom_id` per document, results read in whatever order they arrive, and only failed IDs resubmitted. A batch can't run a tool loop inside a request, so a validation retry goes back to the normal path. On the real Message Batches API this costs half price, takes up to 24 hours and has no latency promise. This stand-in has none of those three properties, so read it for the workflow.

```
uv run extract.py --batch
uv run python -m json.tool evidence/batch.json
```

8. Read `evidence/accuracy.json`. Then break something: change DOC-04's printed total in `fixtures/invoices.txt`, rerun the sync path, and watch `by_document_type.invoice` fall while `receipt` and `credit_note` stay where they were. That is the aggregate-masks-a-segment lesson.

```
uv run extract.py && uv run python -m json.tool evidence/accuracy.json
```

© 2026 Tertiary Infotech Academy · UEN 201200696W

9. Run the verifier.

```
uv run verify.py
```

Labs that run on this simulator

Open this simulator in your browser: from the `labs` folder run `uv run serve.py`, then pick Simulator 6. In Claude Code, type `/lab` and the simulator number. A simulator is the system. A lab is one exam situation you run on it. Run any of these from the `labs` folder with `uv run run_lab.py <lab>`; each has its own PDF under its topic folder. An asterisk marks a lab run in class.

Lab	Task	Source	What it proves	Command
4	1.2	Practice Q36	Same brief to every subagent, same findings four times	`uv run extract.py --case UC-36`
11	1.4	Practice Q12	Claude guesses a field and submits without validating	`uv run extract.py --case UC-12`
15*	1.6	Practice Q31	60-page packets: section passes first, reconcile second	`uv run extract.py --case UC-31`
18*	2.1	Practice Q11	One analyze_document tool does three jobs badly	`uv run extract.py --case UC-11`
42	4.3	Task statement	Know your tool_choice options, and check what came back	`uv run extract.py --tool-choice forced --max-docs 2`
44*	4.4	Task statement	Retry when the format is wrong, stop when the data is absent	`uv run extract.py`
47	4.5	Practice Q32	Nightly job on Batches, upload flow stays synchronous	`uv run extract.py --case UC-32`
57	5.4	Practice Q22	Keep discovery out of the main thread	`uv run extract.py --case UC-22`
60*	5.5	Task statement	The aggregate masks a segment: break it and watch	`uv run extract.py`

Expected evidence

- `evidence/extraction.json`: the `tool_choice` observation, per-document attempts with the model, stop reason and token counts, validation errors, retry decisions, routing reasons.
- `evidence/accuracy.json`: overall, by field, by document type, and the stratified audit sample.
- `evidence/batch.json`: batch IDs, submitted and succeeded `custom_id`s, the failed IDs and their resubmission.
- `evidence/cases/UC-<n>.json`: one file per practice-test use case.
- `evidence/verification.txt`: twenty-two PASS lines when every use case has been run.

Acceptance checks

- Every extraction is a `tool_use` block from `extract_invoice`; prose answers are treated as failures.
- A missing due date is `null`, never a guessed date.
- Baht becomes `other` with `currency_detail` THB; an unstated currency becomes `unclear`.

© 2026 Tertiary Infotech Academy · UEN 201200696W

- The GST receipt is flagged by `calculated_total` versus `stated_total`, not by the schema.
- A retry carries the original document, the failed extraction and the exact error; a retry is not attempted when `missing_information` names the gap.
- Resubmission lists only failed `custom_id`s and states the modification.

Exam link

Practice-test questions 11, 12, 22, 31, 32 and 36. Task statements 4.3 (tool_use schemas, strict tools, tool_choice any and forced, nullable fields, other + detail, normalisation rules), 4.4 (retry with error feedback, limits of retry, calculated vs stated total, conflict_detected), 4.5 (Message Batches API, custom_id, resubmitting failures, sync versus batch), 5.5 (field-level confidence, human review routing, stratified sampling, accuracy by type and field).

© 2026 Tertiary Infotech Academy · UEN 201200696W